

Hybrid Intelligent Systems

CSC3034 Computational Intelligence

Dr Tang Tiong Yew

Introduction

Neuro-Fuzzy Systems

ANFIS: Adaptive Neuro-Fuzzy Inference System

Evolutionary Neural Networks

Fuzzy Evolutionary Systems

Previously we have considered several intelligent technologies, including evolutionary computation, artificial neural network, and fuzzy system.

A **hybrid intelligent system** is one that combines at least two intelligent technologies.

The combination of evolutionary computation, artificial neural networks, and fuzzy systems forms the core of **soft computing**.

Traditional or 'hard' computing crisp values, or numbers

Soft computing soft values, or fuzzy sets

Soft computing is capable of operating with uncertain, imprecise and incomplete information in a manner that reflects human thinking.

How do we then combine different intelligent technologies?

...British police, German mechanics, French cuisine,
Swiss banking, and Italian love...

is a good hybrid

...British cuisine, German police, French mechanics,
Swiss love, and Italian banking...

is a bad hybrid

Likewise, a hybrid intelligent system can be good or bad.

We need to understand the strengths and weaknesses of different components in order to create a good hybrid system.

Comparisons of paradigms

	EC/GA	NN	FS
Knowledge representation			
Uncertainty tolerance			
Imprecision tolerance			
Adaptability			
Learning ability			
Explanation ability			
Knowledge discovery and data mining			
Maintainability			

 <  <  < 

We will be looking at

- ▶ a neuro-fuzzy system,
- ▶ an adaptive neuro-fuzzy inference system (ANFIS),
- ▶ an evolutionary neural network, and
- ▶ a fuzzy evolutionary system.

Introduction

Neuro-Fuzzy Systems

ANFIS: Adaptive Neuro-Fuzzy Inference System

Evolutionary Neural Networks

Fuzzy Evolutionary Systems

Neuro-fuzzy systems are intelligent systems that combine **neural networks** and **fuzzy logic**.

Neural networks	Fuzzy logic
low-level computational structures	high-level reasoning
good with raw data	using linguistic information from domain expert
opaque to the users	cannot learn or adjust to new environment

An integrated neuro-fuzzy system combines

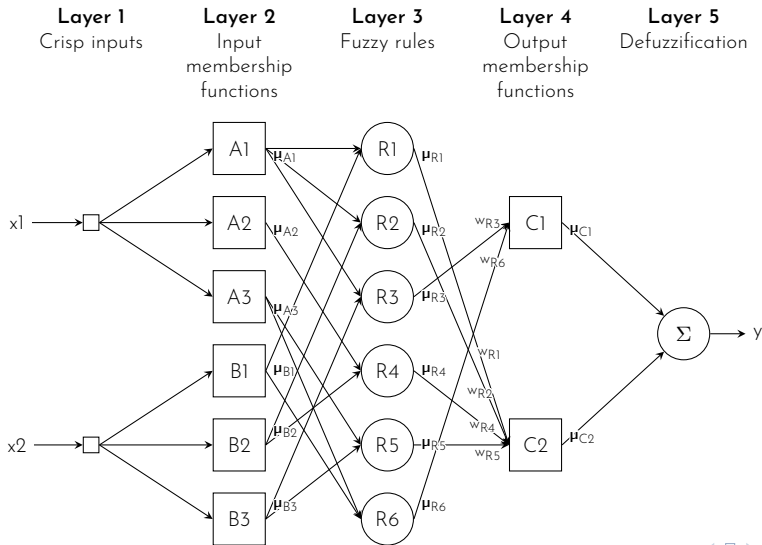
- ▶ parallel computation and learning abilities of neural networks
- ▶ with the human-like knowledge representation and explanation abilities of fuzzy systems.

Neural networks become more transparent

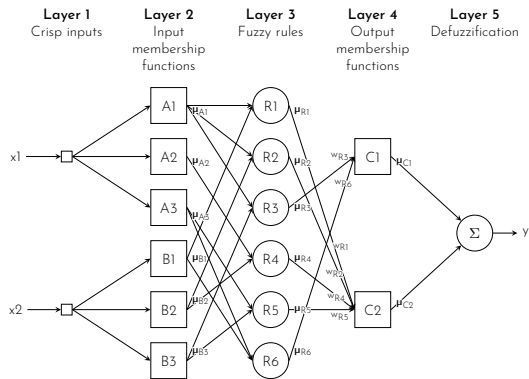
Fuzzy systems gain the capability of learning.

- ▶ A neuro-fuzzy system is a neural network that is functionally equivalent to a fuzzy inference model.
- ▶ It can be trained to develop IF-THEN fuzzy rules and determine membership functions for input and output variables of the system.

Structure of a neuro-fuzzy system

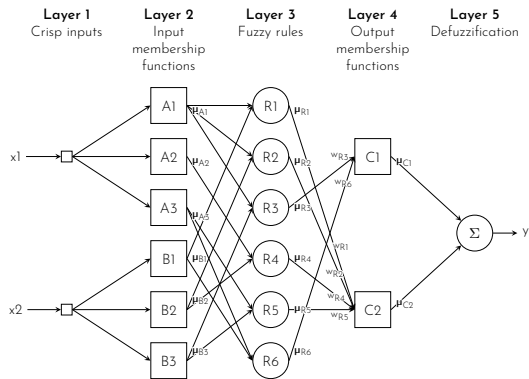


Structure of a neuro-fuzzy system



- ▶ two inputs - x_1 and x_2
- ▶ one output - y
- ▶ input x_1 - fuzzy sets A1, A2, and A3
- ▶ input x_2 - fuzzy sets B1, B2, and B3
- ▶ output y - fuzzy sets C1 and C2
- ▶ based on Mamdani fuzzy inference model

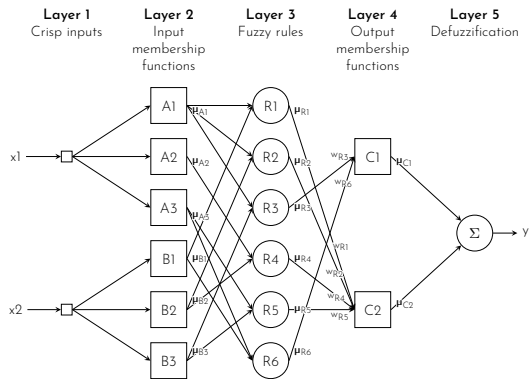
Structure of a neuro-fuzzy system



Layer 1: input layer

- External crisp signals are transmitted directly to the next layer

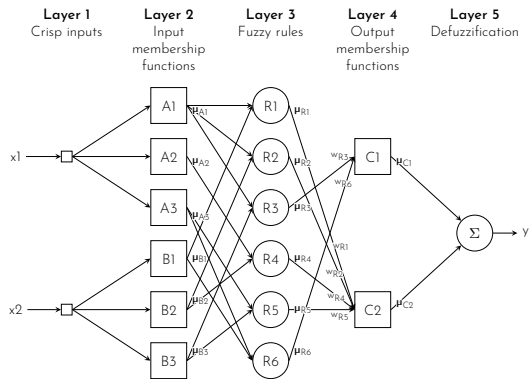
Structure of a neuro-fuzzy system



Layer 2: input membership layer or fuzzification layer

- ▶ The neurons represent fuzzy sets used in the antecedents of the fuzzy rules.
- ▶ This layer determines the degree of membership of the inputs.
- ▶ The activation function is set to the function that specifies the neuron's fuzzy set, for example, a triangular membership function.

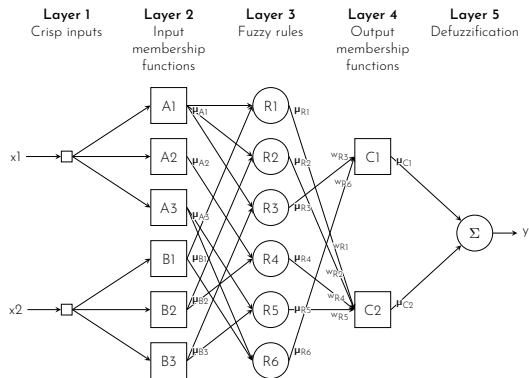
Structure of a neuro-fuzzy system



Layer 3: fuzzy rules

- ▶ Each neuron corresponds to a single fuzzy rule.
- ▶ A fuzzy rule neuron receives inputs from the fuzzification neurons that represent fuzzy sets in the rule antecedents.
- ▶ Multiple rule antecedents are evaluated using the intersection operation (AND).

Structure of a neuro-fuzzy system



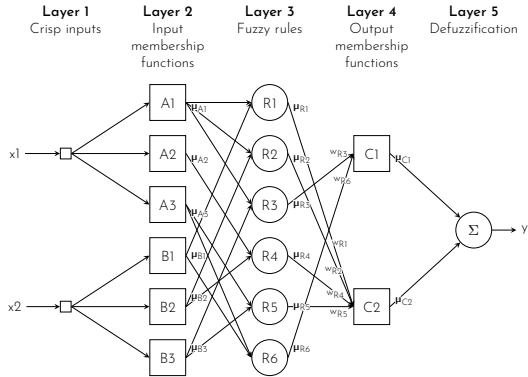
Layer 3: fuzzy rules (continued)

- Normally in a neuro-fuzzy system, intersection is implemented by the product operator instead of min operator.

$$y_i^{(3)} = x_{1i}^{(3)} \times x_{2i}^{(3)} \times \dots \times x_{ki}^{(3)}$$

For example, $y_{R1}^{(3)} = \mu_{A1} \times \mu_{B1} = \mu_{R1}$ (μ_{R1} is the firing strength of fuzzy rule R1)

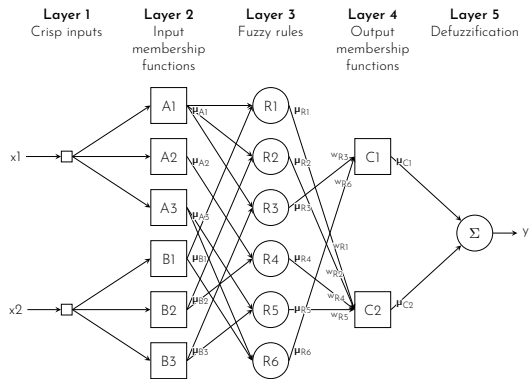
Structure of a neuro-fuzzy system



Weights between layers 3 and 4

- ▶ The weights are the normalised degrees of confidence of the fuzzy rules.
- ▶ It represents the importance or confidence of a fuzzy rule.

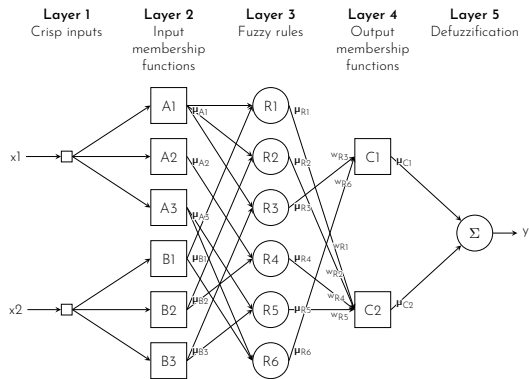
Structure of a neuro-fuzzy system



Layer 4: output membership layer

- ▶ The neurons represent fuzzy sets used in the rule consequent.
- ▶ An output membership neuron receives inputs from the corresponding fuzzy rule neurons and combines them using the fuzzy operation union (OR).

Structure of a neuro-fuzzy system



Layer 4: output membership layer (continued)

- The fuzzy operation union (OR) commonly implemented using the probabilistic OR in neuron-fuzzy systems (instead of max operator).

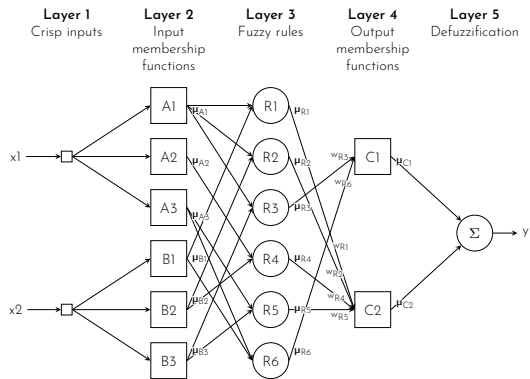
$$y_i^{(4)} = \text{probor}(x_{1i}^{(4)}, x_{2i}^{(4)}, \dots, x_{li}^{(4)})$$

$$= x_{1i}^{(4)} \oplus x_{2i}^{(4)} \oplus \dots \oplus x_{li}^{(4)}$$

For example, $y_{C1}^{(4)} = \mu_{R3} \oplus \mu_{R6} = \mu_{C1}$

$$= \mu_{R3} + \mu_{R6} - \mu_{R3} \times \mu_{R6}$$

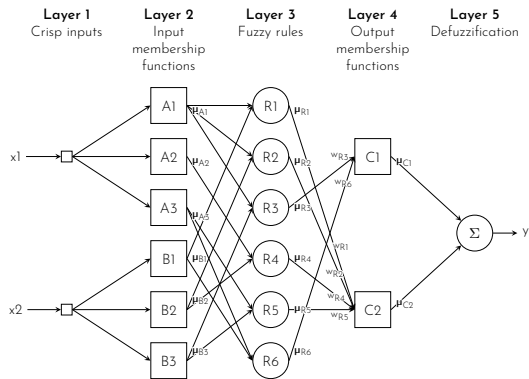
Structure of a neuro-fuzzy system



Layer 4: output membership layer (continued)

- ▶ The activation function in an output membership neuron is the membership function of the corresponding fuzzy set.
- ▶ Therefore the activation function of the neuron C1 is clipped by the firing strength of the rule μ_{C1} .

Structure of a neuro-fuzzy system

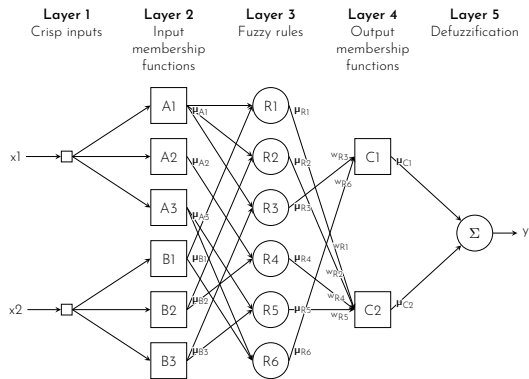


Layer 5: defuzzification layer

- ▶ Each neuron in this layer represents a single output of the neuro-fuzzy system.
- ▶ It combines the output fuzzy sets clipped by the firing strengths of the rules into a single fuzzy set.
- ▶ The combined fuzzy set is defuzzified to find the crisp output of the system.
- ▶ Defuzzification can be achieved using the centroid technique.

Neuro-fuzzy systems

What does the algorithm modify during learning?

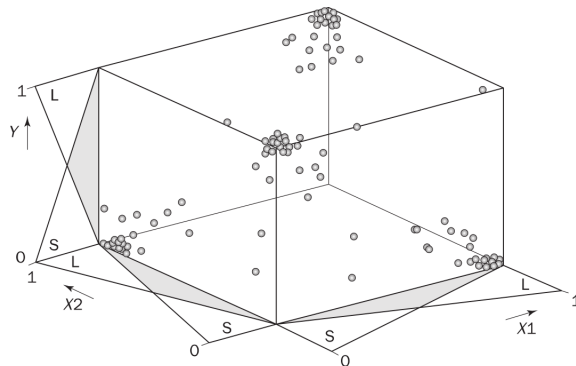


- parameters in membership functions
- normalised degree of confidence (weights between layers 3 and 4)

Since a neural-fuzzy system is essentially a multilayer neural network, we can apply standard learning algorithms developed for neural network, including the back-propagation algorithm.

Example 1

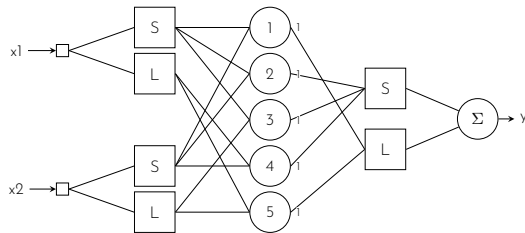
Let us consider the 100 training patterns in the following figure.



- ▶ two inputs: x_1 and x_2
- ▶ one output: y
- ▶ each variable is represented by two linguistic values: small (S) and large (L)

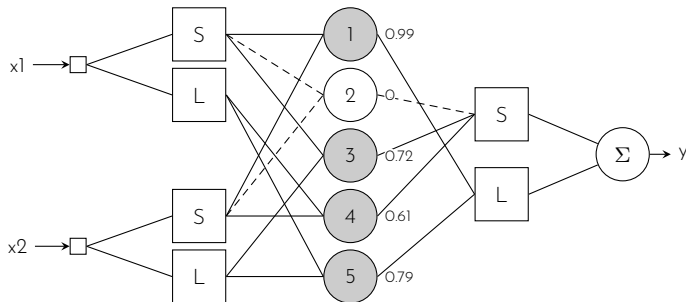
Example 1

- ▶ Suppose that a domain expert provides five rules.
- ▶ It is possible that the expert might make mistakes.
- ▶ A neuro-fuzzy system should be able to identify the bad/wrong rules by training on the training patterns.
- ▶ The initial state of the neuro-fuzzy system is designed such that it follows the specification of the domain expert.

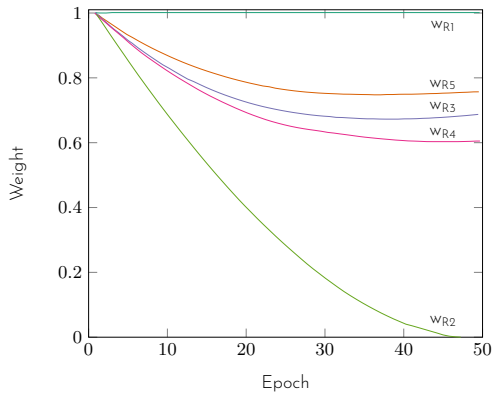


Example 1

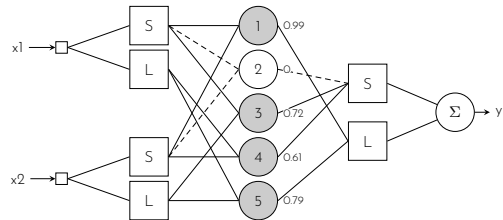
- ▶ The neuro-fuzzy system undergoes training using the back-propagation algorithm to adjust the weights and to modify the input and output membership functions.
- ▶ The training continues until the sum of squared errors is less than 0.001.



Example 1

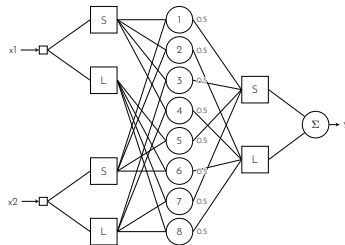


► Rule 2 is found to be false.

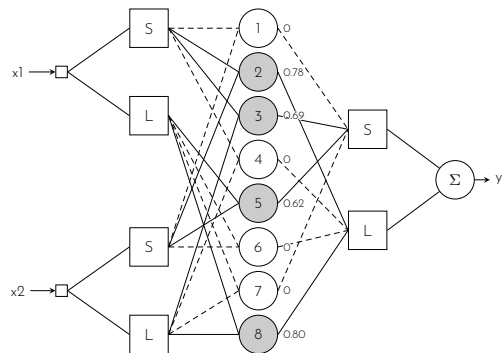
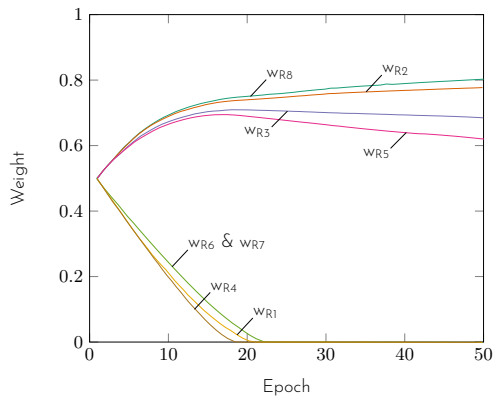


Example 1

- ▶ Experts might provide wrong rules, and they might have left out rules.
- ▶ To remove/reduce dependency on experts, we can set up a generic neuro-fuzzy system and let the system to learn the rules from the training patterns automatically.
- ▶ As we have two inputs and one output, each with two fuzzy subsets, a neuro-fuzzy system with $2 \times 2 \times 2 = 8$ rules can be set up.



Example 1



- The same set of four fuzzy rules are identified.

Introduction

Neuro-Fuzzy Systems

ANFIS: Adaptive Neuro-Fuzzy Inference System

Evolutionary Neural Networks

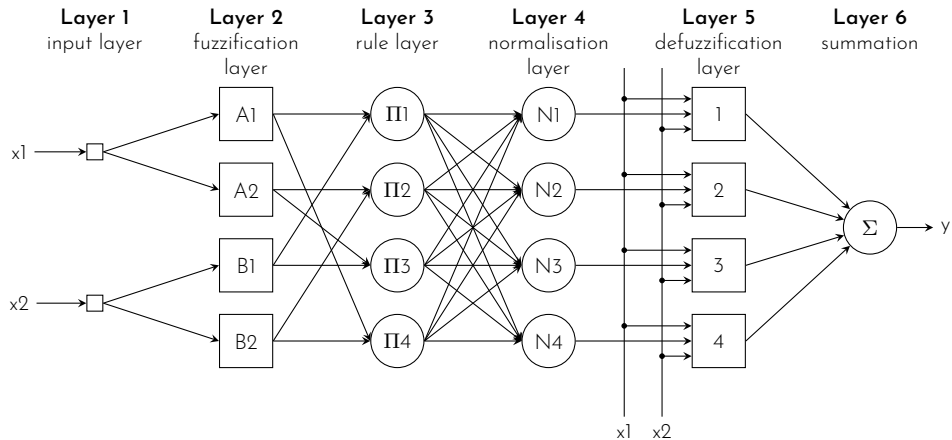
Fuzzy Evolutionary Systems

ANFIS: Adaptive neuro-fuzzy inference system

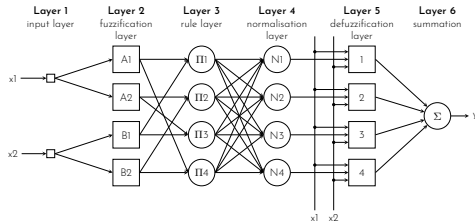
- ▶ The neuro-fuzzy system discussed previously implements the Mamdani fuzzy inference model.
- ▶ ANFIS is the neuro-fuzzy system that implements a Sugeno fuzzy inference model.

Recall: What is the difference between Mamdani inference and Sugeno inference?

Structure of an ANFIS



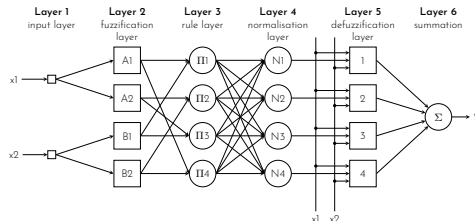
Structure of an ANFIS



Example system:

- ▶ two inputs, each represented by two fuzzy sets
- ▶ one output, represented by a first-order polynomial

Structure of an ANFIS



Example system with four rules:

Rule 1:

IF x_1 is A_1
AND x_2 is B_1
THEN $y = f_1 = k_{10} + k_{11}x_1 + k_{12}x_2$

Rule 2:

IF x_1 is A_2
AND x_2 is B_2
THEN $y = f_2 = k_{20} + k_{21}x_1 + k_{22}x_2$

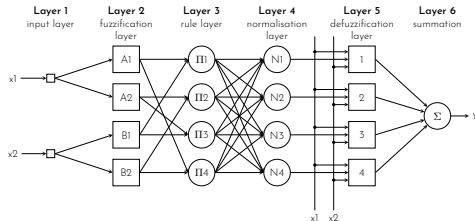
Rule 3:

IF x_1 is A_2
AND x_2 is B_1
THEN $y = f_3 = k_{30} + k_{31}x_1 + k_{32}x_2$

Rule 4:

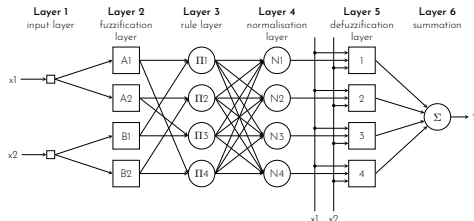
IF x_1 is A_1
AND x_2 is B_2
THEN $y = f_4 = k_{40} + k_{41}x_1 + k_{42}x_2$

Structure of an ANFIS



Layer 1: input layer

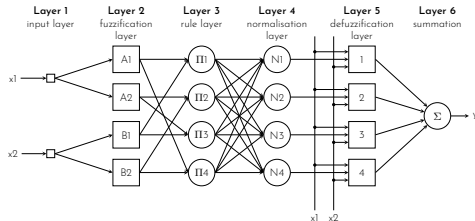
- pass the external crisp signals to layer 2.



Layer 2: fuzzification layer

- Neurons in this layer perform fuzzification.
- The activation functions are the input membership functions.
- In the original model implemented by Jang, fuzzification neurons have bell activation functions.

$$y_i^{(2)} = \frac{1}{1 + \left(\frac{x_i^{(2)} - a_i}{c_i} \right)^{2b_i}}$$

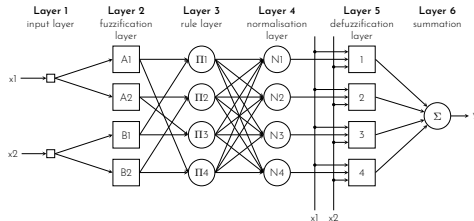


Layer 3: rule layer

- ▶ Each neuron corresponds to a single Sugeno-style fuzzy rule.
- ▶ The rule antecedents are combined with AND, and in ANFIS, the product operator is used.

$$y_i^{(3)} = \prod_{j=1}^k x_{ji}^{(3)}$$

For example, $y_{\Pi_1}^{(3)} = \mu_{A1} \times \mu_{B1} = \mu_1$
(μ_1 is the firing strength or truth value of rule 1.)

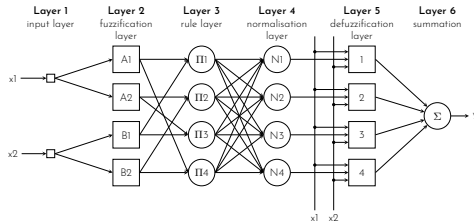


Layer 4: normalisation layer

- ▶ Each neuron receives inputs from **all** neurons in the rule layer.
- ▶ The neuron calculates the normalised firing strength of a given rule.

$$y_i^{(4)} = \frac{x_{ii}^{(4)}}{\sum_{j=1}^n x_{ji}^{(4)}} = \frac{\mu_i}{\sum_{j=1}^n \mu_j} = \bar{\mu}_i$$

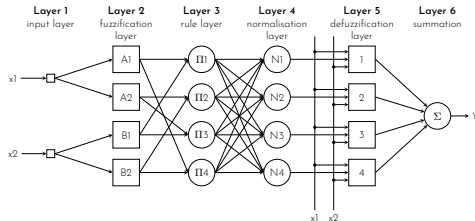
$$\text{For example, } y_{N1}^{(4)} = \frac{\mu_1}{\mu_1 + \mu_2 + \mu_3 + \mu_4} = \bar{\mu}_1$$



Layer 5: defuzzification layer

- ▶ Each neuron receives inputs from corresponding neuron in the normalisation layer, and also receives initial inputs, x_1 and x_2 . neurons in the rule layer.
- ▶ A neuron calculates the weighted consequent value of a given rule as

$$\begin{aligned} y_i^{(5)} &= x_i^{(5)} [k_{i0} + k_{i1}x_1 + k_{i2}x_2] \\ &= \bar{\mu}_i [k_{i0} + k_{i1}x_1 + k_{i2}x_2] \end{aligned}$$



Layer 6: summation neuron

- This layer is represented by a single summation neuron.
- It calculates the sum of outputs of all defuzzification neurons and produces the overall ANFIS output

$$y = \sum_{i=1}^n x_i^{(6)}$$

$$= \sum_{i=1}^n \bar{\mu}_i [k_{i0} + k_{i1}x_1 + k_{i2}x_2]$$

What are the parameters an ANFIS modified during the training phase?

Introduction

Neuro-Fuzzy Systems

ANFIS: Adaptive Neuro-Fuzzy Inference System

Evolutionary Neural Networks

Fuzzy Evolutionary Systems

Neural networks are commonly trained with back-propagation learning algorithm.

Issues:

- ▶ Back-propagation learning algorithm cannot guarantee an optimal solution.
- ▶ The optimal topology for the neural network is difficult to identify, often chosen by means of heuristics (trial and error + experience).

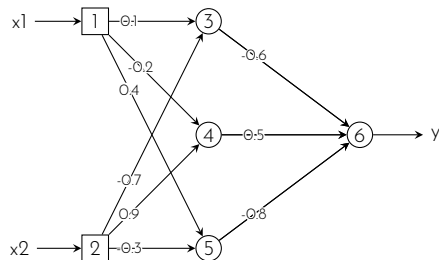
Result: Sub-optimal weights and topology

An evolutionary neural network uses **genetic algorithm** to find the optimal weights and topology of a **neural network**.

- ▶ Multiple neural networks with different weights and topology are generated.
- ▶ The parameters (weights and topology) of each neural network are encoded as a chromosome for the GA.
- ▶ GA optimises the chromosomes by means of crossover and mutation.
- ▶ As a result we get an 'evolved' neural network.

As example,

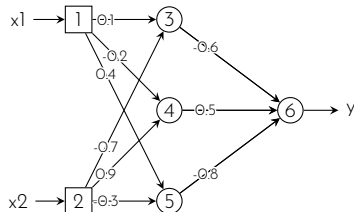
	From neuron					
	1	2	3	4	5	6
To neuron	1	0	0	0	0	0
	2	0	0	0	0	0
	3	0.1	-0.7	0	0	0
	4	-0.2	0.9	0	0	0
	5	0.4	-0.3	0	0	0
	6	0	0	-0.6	0.5	-0.8



Initial weights are randomly chosen with small interval, say $[-1,1]$.

Evolutionary neural networks

		From neuron					
		1	2	3	4	5	6
To neuron	1	0	0	0	0	0	0
	2	0	0	0	0	0	0
	3	0.1	-0.7	0	0	0	0
	4	-0.2	0.9	0	0	0	0
	5	0.4	-0.3	0	0	0	0
	6	0	0	-0.6	0.5	-0.8	0



0.1	-0.7	-0.2	0.9	0.4	-0.3	-0.6	0.5	-0.8
-----	------	------	-----	-----	------	------	-----	------

Each row

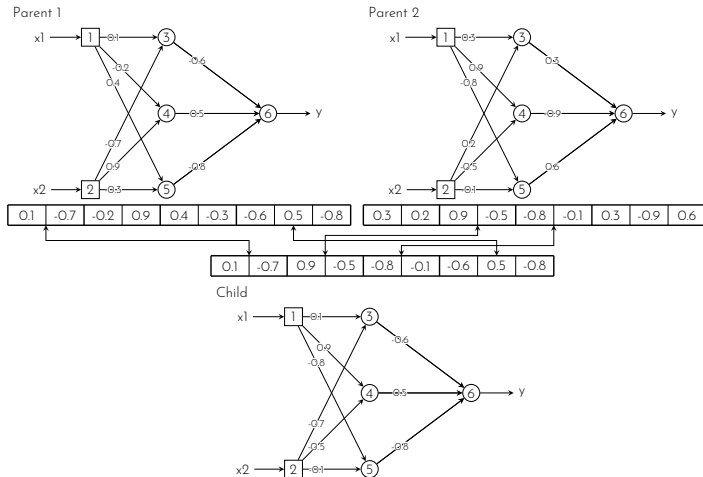
- ▶ represents a group of all the incoming weighted links to a single neuron.
- ▶ can be thought of as a functional building block of the network.
- ▶ should stay together as one genetic material passing from one generation to the next.

What is the fitness function to evaluate the chromosome's performance?

- ▶ The fitness function will estimate the performance of a given neural network.
 1. assign the weights of the chromosome to the neural network,
 2. present the training data to the neural network,
 3. identify the sum of squared errors, and
 4. use the reciprocal of the sum of squared errors as fitness function.
- ▶ The aims of the GA is to find the weights that minimises the sum of squared errors.

How do crossover and mutation take place?

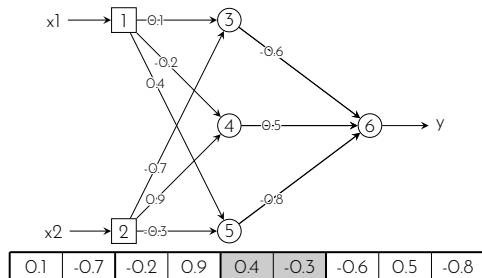
Crossover



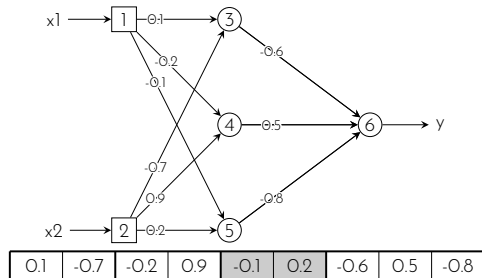
Evolutionary neural networks

Mutation

Original network



Mutated network



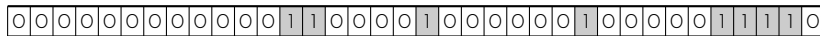
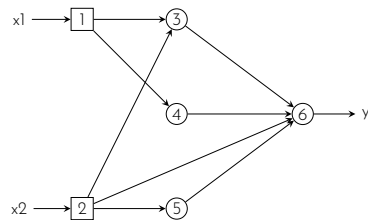
The selected gene will be mutated by adding a small random value between -1 and 1.

At this stage the GA is only used to optimise the weights, the structure of the network is assumed fixed.

How do we formulate (encode) the chromosome to allow GA to assist in identifying the optimal network architecture?

The simplest method is the direct encoding.

		From neuron					
		1	2	3	4	5	6
To neuron	1	0	0	0	0	0	0
	2	0	0	0	0	0	0
	3	1	1	0	0	0	0
	4	1	0	0	0	0	0
	5	0	1	0	0	0	0
	6	0	1	1	1	1	0



The direct encoding can only be applied to feedforward networks with a fixed number of neurons. However, it demonstrates how a connection topology is evolved.

Evolutionary computation can be used to optimise

- ▶ weights,
- ▶ topology,
- ▶ transfer functions, and
- ▶ suitable input variables

of a neural network.

Introduction

Neuro-Fuzzy Systems

ANFIS: Adaptive Neuro-Fuzzy Inference System

Evolutionary Neural Networks

Fuzzy Evolutionary Systems

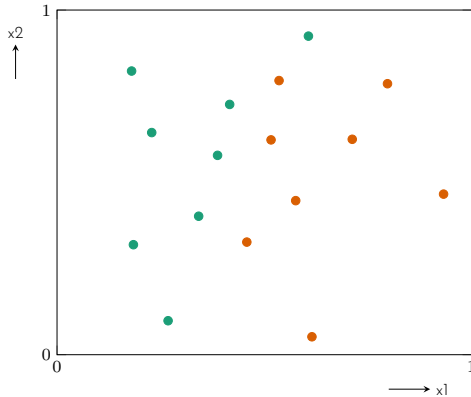
A fuzzy evolutionary system is a fuzzy inference system of which the fuzzy rules are optimally selected using a genetic algorithm.

The main steps in a fuzzy evolutionary system are

1. generating a set of rules and
2. selecting the optimal rules using genetic algorithm.

Fuzzy evolutionary systems

Consider the training patterns for a classification problem with two inputs and one output.

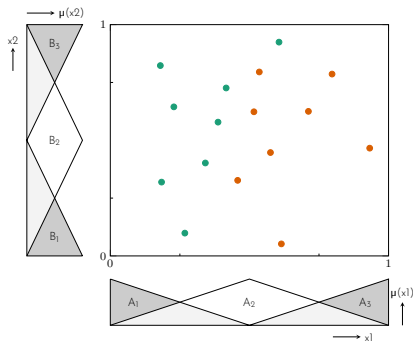


Grid-type fuzzy partition and **multiple fuzzy rule tables** can be used to generate a set of IF-THEN fuzzy rules from numerical data.

Fuzzy evolutionary systems

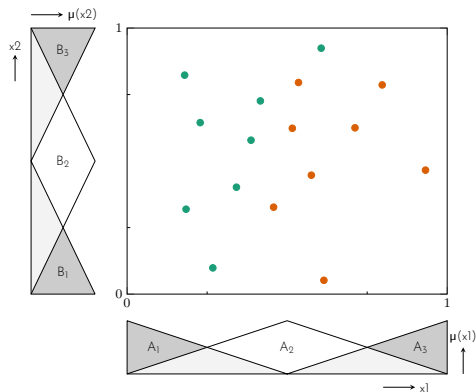
Grid-type fuzzy partition

First we consider that both inputs (x_1 and x_2) have three linguistic values each.



Each fuzzy subspace can have only one fuzzy IF-THEN rule.

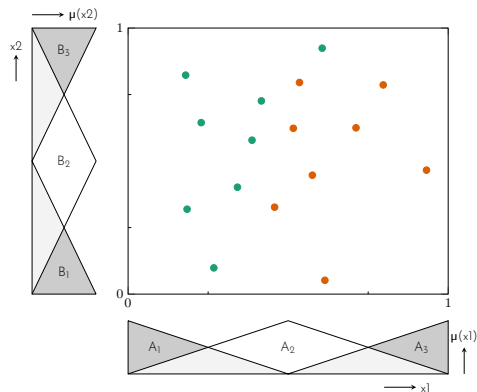
Fuzzy evolutionary systems



In each fuzzy subspace, the rule antecedent uses the AND operator to construct the rule.

How do we identify the rule consequent of a fuzzy subspace?

Fuzzy evolutionary systems



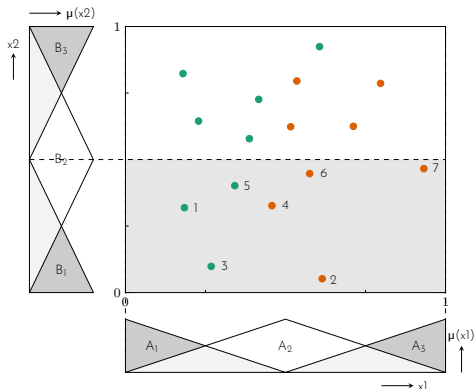
To determine the rule consequent of a fuzzy subspace, we first **calculate the strength of each class of training patterns in every fuzzy subspace**.

The strength of a class C_n is calculated by

$$\beta_{A_i B_j}^{C_n} = \sum \mu_{A_i} \times \mu_{B_j}$$

for all points in the fuzzy subspace that belongs to class C_n .

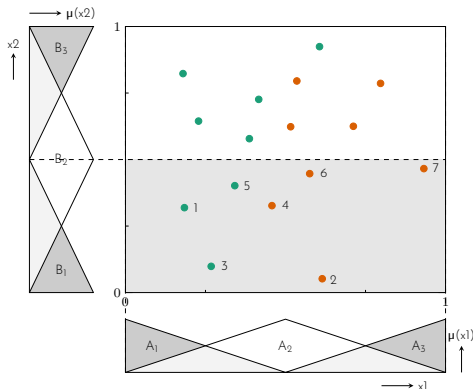
For example, consider the fuzzy subspace A_2B_1 .



Points 1, 3, and 5 are in class 1, and points 2, 4, 6, and 7 are in class 2.

We denote the points as p_1 to p_7 .

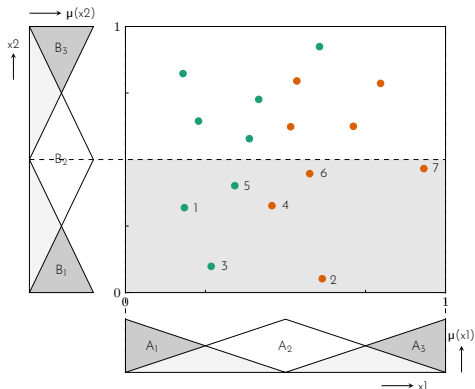
For example, consider the fuzzy subspace A_2B_1 .



Strength of class 1 in subspace A_2B_1 is

$$\begin{aligned}\beta_{A_2B_1}^{C_1} &= \mu_{A_2p_1} \times \mu_{B_1p_1} + \mu_{A_2p_3} \times \mu_{B_1p_3} \\ &\quad + \mu_{A_2p_5} \times \mu_{B_1p_5} \\ &= 0.41 \times 0.38 + 0.54 \times 0.81 \\ &\quad + 0.65 \times 0.21 \\ &= 0.73\end{aligned}$$

For example, consider the fuzzy subspace A_2B_1 .



Strength of class 2 in subspace A_2B_1 is

$$\begin{aligned}\beta_{A_2B_1}^{C_2} &= \mu_{A_2p_2} \times \mu_{B_1p_2} + \mu_{A_2p_4} \times \mu_{B_1p_4} \\ &\quad + \mu_{A_2p_6} \times \mu_{B_1p_6} + \mu_{A_2p_7} \times \mu_{B_1p_7} \\ &= 0.75 \times 0.89 + 0.92 \times 0.34 \\ &\quad + 0.87 \times 0.12 + 0.11 \times 0.09 \\ &= 1.09\end{aligned}$$

From the strength of each class, **the rule consequent is determined by the strongest class**, i.e. the class with highest strength.

The certainty factor can then be calculated using

$$CF_{A_i B_j}^{C_m} = \frac{\beta_{A_i B_j}^{C_m} - \beta_{A_i B_j}}{\sum_{n=1}^N \beta_{A_i B_j}^{C_n}} \quad \text{where} \quad \beta_{A_i B_j} = \frac{\sum_{n=1, n \neq m}^N \beta_{A_i B_j}^{C_n}}{N - 1}$$

So the certainty factor of the rule consequent for fuzzy subspace $A_2 B_1$ is

$$CF_{A_2 B_1}^{C_2} = \frac{1.09 - 0.73}{1.09 + 0.73} = 0.1978$$

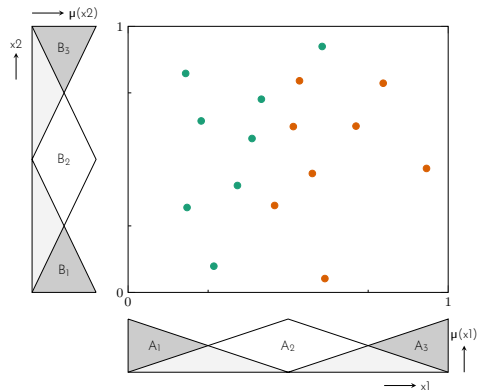
What is a certainty factor?

Certainty factor represents how certain we are that the input patterns in a subspace should be classified under the class. In other words, how certain we are of the rule corresponding to the subspace.

If all training patterns in that fuzzy subspace belongs to the same class, then the certainty factor is maximum.

If the training patterns in the fuzzy subspace belongs to different class, and these classes have similar strengths, the certainty factor is minimum.

Fuzzy evolutionary systems

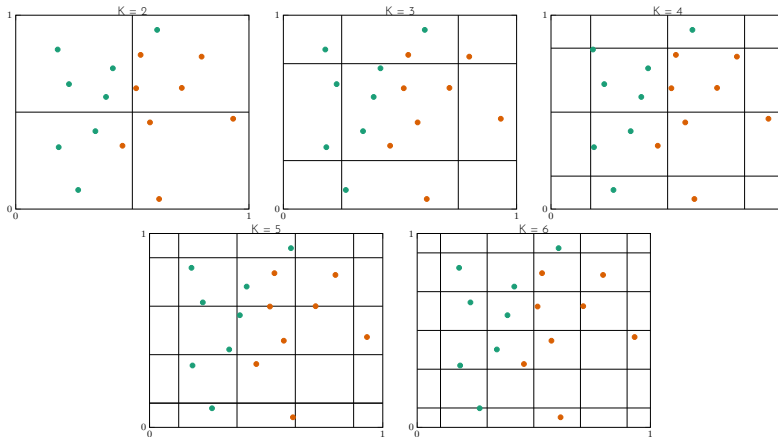


As training patterns are not necessarily distributed evenly, there might be subspaces with mixed training patterns (i.e. low certainty factor), or with small amount of training patterns.

To overcome this, **multiple fuzzy rule tables** are used.

Fuzzy evolutionary systems

Multiple fuzzy rule table



K is the number of fuzzy intervals in each axis/input

As one fuzzy subspace generates one rule, using multiple fuzzy rule table with $K = 2$ to 6, we will get $2^2 + 3^2 + 4^2 + 5^2 + 6^2 = 90$ rules.

At this stage, we can actually identify the class of a new pattern based on the rules. However, to evaluate this using 90 rules is tedious.

GA is used to identify the optimal rules that are significant, and therefore dramatically reducing the number of rules.

To use GA to identify the optimal rule set, we need to encode the problem.

- ▶ A chromosome of 90-bit string is used to represent the 90 fuzzy rules generated using the multiple fuzzy rule tables.
- ▶ Each bit in the string can be: -1, 1, or 0.

- ▶ The purpose of the GA is to select the significant rules and remove non-significant rules.
- ▶ Each chromosome carries the information of whether a rule is selected or removed.
- ▶ Each bit in a chromosome corresponds to a rule.
 - If the bit is 1, the rule is selected.
 - If the bit is -1, the rule is removed.
 - If the bit is 0, the rule is a dummy rule.

A dummy rule is generated when the consequent of the rule cannot be determined, normally due to no training pattern in the fuzzy subspace.

The aims of the GA are

1. to maximise the accuracy of the pattern classification and
2. to minimise the size of a rule set.

Therefore the fitness function is

$$f(S) = w_p \frac{P_s}{P_{\text{all}}} - w_n \frac{N_s}{N_{\text{all}}}$$

where w_p and w_n are weights

P_s is the number of patterns classified correctly

P_{all} is the total number of patterns

N_s is the number of selected fuzzy rules

N_{all} is the number of all fuzzy rules.

$$f(S) = w_p \frac{P_s}{P_{\text{all}}} - w_n \frac{N_s}{N_{\text{all}}}$$

Normally,

$$0 < w_n \leq w_p$$

since classification accuracy is more important than the size of a rule set.

The standard procedure for genetic algorithm apply on fuzzy evolutionary systems.

The end result of a fuzzy evolutionary system is a fuzzy inference model that consists of optimal amount of significant rules.

Hybrid intelligent systems

- ▶ a neuro-fuzzy system
- ▶ an adaptive neuro-fuzzy inference system (ANFIS)
- ▶ an evolutionary neural network
- ▶ a fuzzy evolutionary system



SUNWAY
UNIVERSITY

Thank you

CSC3034 Current Course Update

09 Hybrid Intelligence Systems - handout

Synchronized from the current CSC3034 course site on 14 August 2026.

- Current labs connect core CI algorithms with physical-AI simulation examples.
- Isaac Sim examples use its bundled Python 3.12 environment and current launchers.
- Every simulator-dependent example includes a CPU/Matplotlib learning fallback.

Current materials author: Dr Tang Tiong Yew

See the synchronized lab notes and runnable examples for full instructions.